

2. Sort the People

You are given an array of strings `names`, and an array `heights` that consists of **distinct** positive integers.

Both arrays are of length `n`.

For each index `i`, `names[i]` and `heights[i]` denote the name and height of the i^{th} person.

Return `names` sorted in **descending** order by the people's heights.

Example 1:

Input: `names = ["Mary","John","Emma"], heights = [180,165,170]`

Output: `["Mary","Emma","John"]`

Explanation: Mary is the tallest, followed by Emma and John.

Example 2:

Input: `names = ["Alice","Bob","Bob"], heights = [155,185,150]`

Output: `["Bob","Alice","Bob"]`

Explanation: The first Bob is the tallest, followed by Alice and the second Bob.

Constraints:

- `n == names.length == heights.length`
- `1 <= n <= 103`
- `1 <= names[i].length <= 20`
- `1 <= heights[i] <= 105`
- `names[i]` consists of lower and upper case English letters.
- All the values of `heights` are distinct.

PROGRAM

```
import java.util.Arrays;
```

```
public class Solution {
```

```
    public String[] sortPeople(String[] names, int[] heights) {
```

```
        int n = names.length;
```

```
        Integer[] indices = new Integer[n];
```

```
        for (int i = 0; i < n; i++) {
```

```
            indices[i] = i;
```

```
        }
```

```
        // Sort indices based on descending order of heights
```

```
Arrays.sort(indices, (a, b) -> Integer.compare(heights[b], heights[a]));
```

```
// Reconstruct the sorted names array
```

```
String[] result = new String[n];
```

```
for (int i = 0; i < n; i++) {
```

```
    result[i] = names[indices[i]];

```

```
}
```

```
return result;
```

```
}
```

```
}
```

OUTPUT

The screenshot displays the LeetCode submission interface for the problem "Sort the People". The submission is marked as "Accepted" with 68 out of 68 test cases passed. The runtime is 10 ms, which is faster than 56.97% of other submissions. The memory usage is 46.88 MB, which is less than 93.66% of other submissions. A bar chart visualizes the runtime distribution across various time intervals. The code is written in Java, and the test result shows the input names and heights, and the output is the sorted names array.

```
1 import java.util.Arrays;
```

```
14 // Reconstruct the sorted names array
15 String[] result = new String[n];
16 for (int i = 0; i < n; i++) {
17     result[i] = names[indices[i]];
18 }
19 return result;
20 }
21 }
22 }
23 }
```

Testcase | Test Result

Accepted Runtime: 1 ms

Case 1 Case 2

Input

names = ["Mary", "John", "Emma"]

heights = [180, 165, 170]

Output